

Introductions to Big Data Systems: Homework 8

Francisco Diego Ariel Romero Contreras

法兰西思科

francisco.romero.c@ug.uchile.cl

2025403471

Models

For each instance of partition algorithm will be studied the case for 'small graph 5' which is a graph of 5 vertex with one high degree outgoing vertex. To achieve a good distribution on the small graph will be used a different hash function, it will be:

$$\text{HASH}(x) = (x - 1) \% p$$

This because the graph is too small that the some more general hash function tends to make a misbalanced distribution.

For the rest of cases will be used the Wang 32-bit hash function.

Also will be displayed on tables the result for multiple instance of 'synthesized-1b' to see the results on a bigger graph.

Balanced P-way Edge-cut

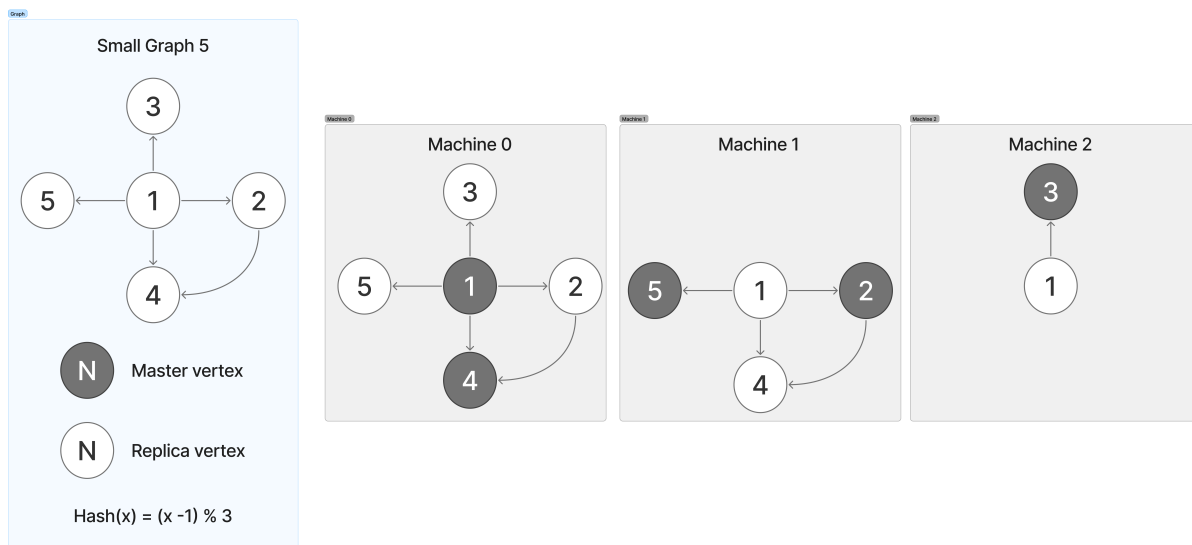
The balance p-way edge-cut core algorithm is as following:

```
For each edge in graph{
    msrc = hash(src) % p;
    mdst = hash(dst) % p;
    if (msrc == mdst){
        Pts[msrc].setMaster.insert(src);
        Pts[msrc].setMaster.insert(dst);
        Pts[msrc].setEdge.insert(edge);
    } else {
        //node source
        Pts[msrc].setMaster.insert(src);
        Pts[msrc].setVertex.insert(dst);
        Pts[msrc].setEdge.insert(edge);
        //node target
        Pts[mdst].setMaster.insert(dst);
        Pts[mdst].setVertex.insert(src);
        Pts[mdst].setEdge.insert(edge);
    }
}
```

Some definition of this pseudo-code:

- Pts: Is a array to access the partitions.
- msrc: Is the machine where the source is assigned
- mdst: Is the machine where the target is assigned
- setMaster: Set of the vertex defined as master.
- setVertex: Set of the vertex defined as mirror.
- setEdge: Set of the edge in the graph.
- p: Number of partitions to do.

Using this algorithm on the graph 'small graph 5' the final distribution is as the image show.



As the vertex 1 is a out-going high degree means that almost every vertex and edge will be on the machine selected as master, in this particular case the partition takes as master one of the vertex with the only edge not connected with vertex 1, so all the graph is on the machine 0. For the rest of machine there is a normal distribution.

	2 partition (0)	2 partition (1)	3 partition (0)	3 partition (1)	3 partition (2)
Number of master vertices	500000	500000	333334	333333	333333
Total number of vertices	868729	877662	737529	755501	710174
Number of edges	3559123	3612627	2671745	2764815	2531082
Number of replicated edges	368729	377662	404195	422168	376841

Table 1: Table with the different stats for 'Balance p-way edge-cut' using the graph 'synthesized-1b' with 2 and 3 partitions

	4 partition (0)	4 partition (1)	4 partition (2)	4 partition (3)
Number of master vertices	250000	250000	250000	250000
Total number of vertices	619763	653409	634887	622244
Number of edges	2036098	2177224	2106755	2045669
Number of replicated edges	369763	403409	384887	372244

Table 2: Table with the different stats for 'Balance p-way edge-cut' using the graph 'synthesized-1b' with 4 partitions

Balanced p-way vertex-cut

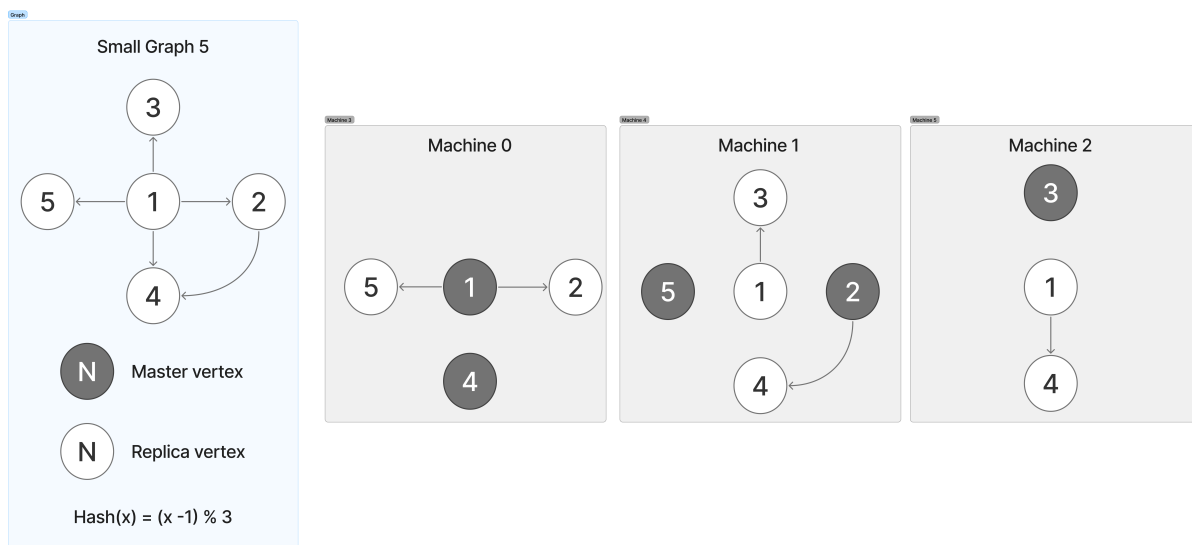
The Balance p-way Vertex-cut core algorithm is as following

```
set i as 0
For each edge in graph{
    medge = hash(i);
    msrc = hash(src);
    mdst = hash(dst);
    Pts[msrc].setMaster.insert(src);
    Pts[mdst].setMaster.insert(dst);
    Pts[medge].setEdge.insert(edge);
    if (msrc != medge){
        Pts[medge].setVertex.insert(src);
    }
    if (mdst != medge){
        Pts[medge].setVertex.insert(dst);
    }
    i++;
}
```

Some definition of this pseudo-code:

- i: Is a iterator that identify the edge
- Pts: Is a array to access the partitions.
- medge: Is the machine where the edge is assigned
- msrc: Is the machine where the source is assigned
- mdst: Is the machine where the target is assigned
- setMaster: Set of the vertex defined as master.
- setVertex: Set of the vertex defined as mirror.
- setEdge: Set of the edge in the graph.

Using this algorithm on the graph 'small graph 5' the final distribution is as the image show.



The distribution among the machines is better than the balanced edge-cut, but this version has more mirror vertex than master vertex, so there is a high duplicated index. Also there are vertices with no edge in each machine.

	2 partition (0)	2 partition (1)	3 partition (0)	3 partition (1)	3 partition (2)
Number of master vertices	500022	499978	332072	333264	334664
Number of total vertices	870993	870560	733965	733578	736018
Number of edges	2390207	2390157	1593297	1591284	1595783

Table 3: Table with the different stats for 'Balance p-way edge-cut' using the graph 'synthesized-1b' with 2 and 3

	4 partition (0)	4 partition (1)	4 partition (2)	4 partition (3)
Number of master vertices	250167	250006	249855	249972
Number of total vertices	631461	629328	631039	629485
Number of edges	1195262	1195103	1194945	1195054

Table 4: Table with the different stats for 'Balance p-way edge-cut' using the graph 'synthesized-1b' with 4 partitions

Balanced p-way Hybrid-cut

The Balance p-way Hybrid-cut core algorithm is as following

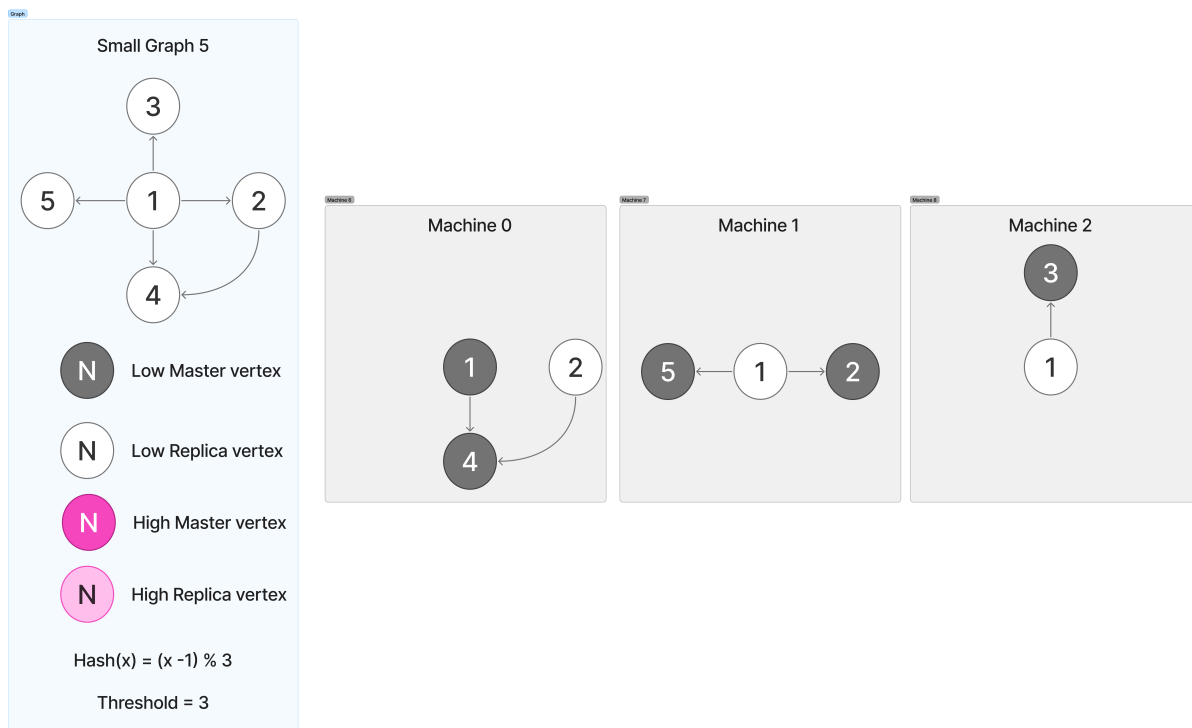
```
For each edge in graph{
    pt = hash(dst);
    if( (degree(dst).size + 1) >= threshold ){
        degree(dst).insert(src);
        Pts[pt].setEdge.insert(edge);
        Pts[pt].setVertex.insert(src);
        Pts[pt].setVertex.insert(dst);
    } else {
        pt = hash(src);
        Pts[pt].setEdge.insert(edge);
        Pts[pt].setVertex.insert(src);
        Pts[pt].setVertex.insert(dst);
    }
}
```

The degree interface is represented different as how it really is for simplicity

Real interface: `degree(dst) = Pts[pt].degree[dst];`

Some definition of this pseudo-code:

- Pts: Is a array to access the partitions.
- pt: Is the machine where the edge is assigned.
- degree(id): is the amount of in-going edges of vertex id.
- setVertex: Set of the vertex defined as mirror.
- setEdge: Set of the edge in the graph.



As there is not vertex with in-going high degree, the distribution among machines is good, this is because the default shuffle is made with the out-going vertex, so the edges with source vertex 1 is distributed by the machines. This algorithm works well in the distribution of high degree edges while maintain a low amount of mirror data.

Greedy Hybrid Vertex-cut

The Balance p-way Hybrid-cut core algorithm is as following

```
for edge in edges matching (source, target){
    bestP = -1
    bestCost = infinity
    bestLoad = infinity
    for p in Partitions{

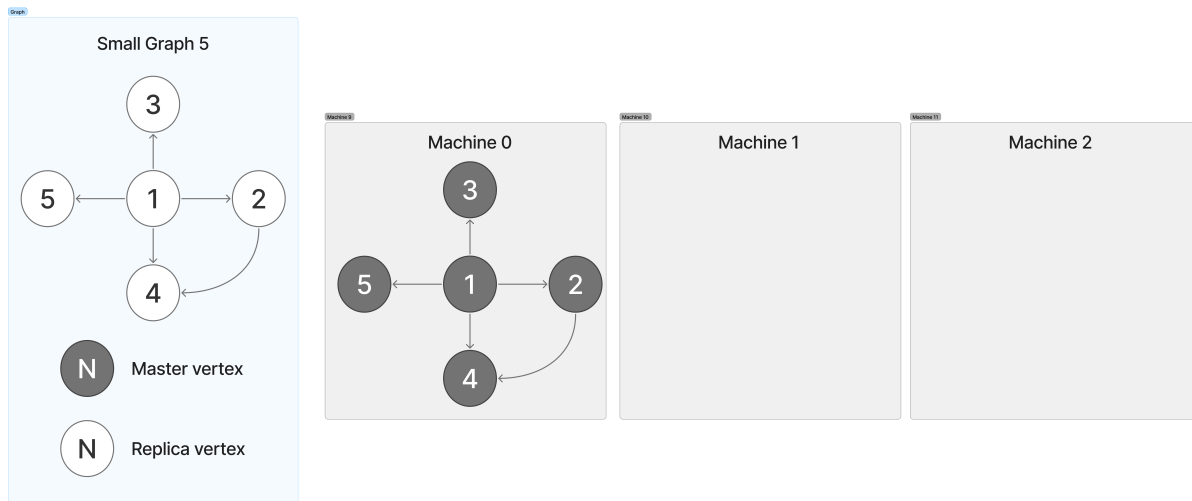
        c1 = u not in p? 1 : 0
        c2 = v not in p? 1 : 0
        cost = c1 + c2
        load = p.setEdge.size()

        if (cost < bestCost){
            bestCost = cost
            bestLoad = load
            bestP = p.id
            clear tiedPartition and add p.id to tiedPartition
        } else if (cost == bestCost and load < bestLoad){
            bestLoad = load
            bestP = p.id
            clear tiedPartition and add p.id to tiedPartition
        } else if (cost == bestCost and load == bestLoad){
            add p.id to tiedPartition
        }
        if (tiedPartition not empty){
            bestP = selcet random from tiedPartition
        }
        p.setEdge.insert(edge)

    }
    itU = vertexMaster.find(u)
    //itU: (vertex, master in partition)
    if (itU not found){
        vertexMaster[u] = bestP
        Pts[bestP].setMaster.insert(u)
    } else {
        uMaster = itU -> second
        if (uMaster inequal bestP){
            Pts[bestP].setVertex.insert(u)
        }
    }
    itV = vertexMaster.find(v)
    //itV: (vertex, master in partition)
    if (itV not found){
        vertexMaster[v] = bestP
        Pts[bestP].setMaster.insert(v)
    } else {
        vMaster = itV -> second
        if (vMaster inequal bestP){
            Pts[bestP].setVertex.insert(v)
        }
    }
}
```

Some definition of this pseudo-code:

- VertexMaster: Is a map with key vertex and value the partition which the vertex is master.
- tiedPartition: List with the tied potential partition.
- cost: Is the answer for the question. How much new vertex i have to add.
- load: Amount of edges in the potential partition.
- bestP: Potential best partition.
- bestCost: Potential cost, depends on bestP.
- bestLoad: Potential load, depends on bestP.
- itU: Is an iterator on the map VertexMaster.
- itV: Is an iterator on the map VertexMaster.



As the vertex 1 have a high out-going degree the algorithm glomerate all the vertices and edges on a single machine. This is not the general case, the usual distribution an equal distribution of the vertex with high degree. But in case of a high dependency over a singular vertex, this algorithm will not be able to perform the partitions.